10 — Duplicate & near-duplicate detection

What it detects: a page that looks the same as a page in a document seen before — a resubmitted invoice, the same quotation re-uploaded under a slightly different vendor name, a re-scan, a re-saved copy. It catches what a byte-level SHA-256 comparison (`documents.file_hash`) cannot: re-saving, re-compressing or trivially editing a file changes its bytes but not how it *looks*.

Code: `services/forensics/duplicate_check.py` (`run_duplicate_check`), run by `tasks/duplicate_check_task.py` (`run_duplicate_check_task`); model `models/document_page_hash.py` (`document_page_hashes`); stores `check_type = duplicate_detection` .

Algorithm

1. Render each page (`render_pdf_pages` , 200 DPI) and compute a **perceptual hash**: `imagehash.phash` (64-bit, `hash_size = 8`) of the RGB page image → a 16-character hex string, one per page (`compute_page_hashes`).
2. `find_matches` loads **every stored page hash of every other document of the same company** (joined to filename, case id and case number) and, for each new page, finds the single **closest** prior page whose Hamming distance is ≤ `DUPLICATE_HASH_HAMMING_THRESHOLD` (default **5** of 64 bits — deliberately conservative: near- identical resubmissions, not merely similar layouts such as two invoices from one template). One match per new page at most.
3. `store_page_hashes` writes this document's own hashes **regardless of outcome**, so future uploads can match against it — as an upsert, one row per (document, page), so a redelivered task can't duplicate them. (The hash is a side effect of the check; there is no separate step.)
4. `result` is `flag` if any page matched, else `pass` .

Match details (filename, case number) are captured **at check time**, not resolved later, so a finding stays readable even if the matched document's case is one the current viewer has never opened.

Output shape (real)

```
{
  "result": "flag",
  "details": [{
    "finding": "near_duplicate_page",
    "severity": "high",
    "page": 1,
    "description": "Page 1 is a near-identical match (perceptual hash Hamming distance 0) to page 1 of
\"Sample15_Arabic_Case_Evidence.pdf\" in case CASE-8D8C00E1 — a possible duplicate or near-duplicate
resubmission.",
    "data": {
      "matched_document_id": "uuid", "matched_document_filename": "Sample15_Arabic_Case_Evidence.pdf",
      "matched_case_id": "uuid", "matched_case_number": "CASE-8D8C00E1",
      "matched_page": 1, "distance": 0
    }
  }]
}
```

Same `{finding, severity, description, page, data}` shape as ELA/copy-move (no `bounding_box` — the whole page is the match), so the shared findings-list UI renders it. In the UI the finding links to the matched case.

`document_page_hashes` : `id` , `company_id` , `document_id` (FK), `page_number` (1-based), `phash` (`varchar(32)` , indexed), timestamps; unique on (`document_id` , `page_number`).

Known limitations

- **It matches against everything the company ever stored**, including your own test uploads. Re-uploading the same sample file in a new case (or uploading the two halves of one case that share a page) **will flag**. This is correct behaviour, and the most common source of "unexpected" flags during testing.
- **Never across companies.** The same invoice submitted to two different client companies is not detected: matching across tenants would reveal another company's filenames and case numbers. This is a deliberate privacy choice.
- **Same-case matches count** (see the note at the top) — despite the rule's `cross_case` name.
- Full-scan comparison per page: cost grows linearly with the number of stored pages. There is no index (e.g. BK-tree/LSH); fine for thousands of pages, not millions.
- pHash is sensitive to layout, not content: two documents that differ only in a changed amount will hash almost identically (that is the intent, and why it is high severity), whereas a heavy crop or rotation can defeat it.
- No cleanup: hashes of documents whose cases are closed stay forever. Two uploads processed at exactly the same moment by parallel forensics processes may not see each other.
- PDF only (as are uploads).

Risk rules fed

`rule_id`	Finding	Weight	Severity
`duplicate.cross_case_match`	`near_duplicate_page`	40	high

This is the heaviest single seeded rule: with the default thresholds (medium ≥ 30, high ≥ 60) one duplicate page alone makes a case **medium** risk; with any other signal it approaches **high**.